

1. Proje Açıklaması ve Hedefleri

Bu çalışmanın amacı, yerel ortamda geliştirilen bir projenin, bulut bilişim prensiplerine uygun olarak taşınması, dağıtılması ve canlıya alınması sürecinin deneyimlendirilmesidir.

Bulut ortamında taşınmak üzere seçilen uygulama, **ASP.NET Core MVC** mimarisi kullanılarak geliştirilmiş bir **Proje ve Görev Takip Sistemidir**. Uygulama, takımların projelerini yönetmesine, görev ataması yapmasına ve süreçleri takip etmesine olanak tanır. Proje, “basit bir web uygulaması veya Veritabanı sunucusu” kriterlerini karşılamaktadır.

2. Bulut Platformu ve Teknoloji Seçimi

Projenin dağıtımı için bulut servis sağlayıcısı olarak **Render.com** platformu tercih edilmiştir. Bu seçimin yapılmasında ödev metninde belirtilen kriterler ve aşağıdaki teknik gerekçeler etkili olmuştur:

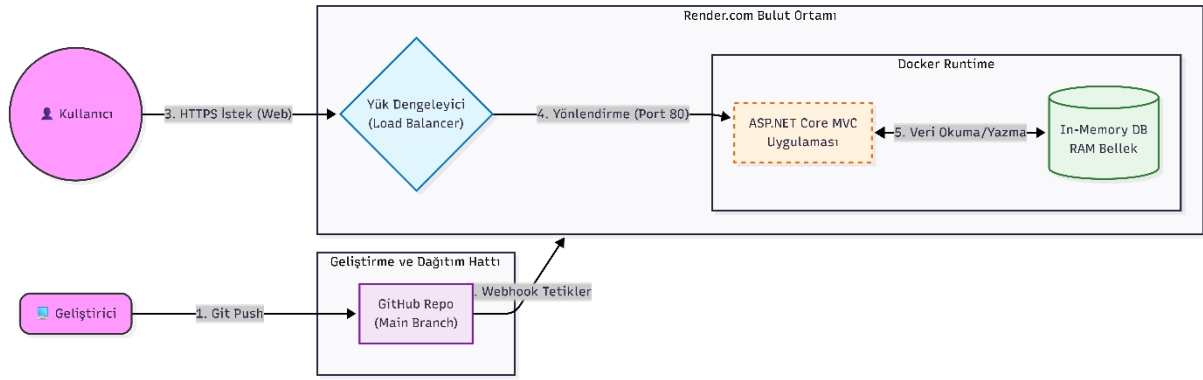
- **Maliyet ve Risk Yönetimi:** Ödev yönergesinde vurgulanan “ücretlendirme ile karşılaşma riski” göz önüne alınarak, kredi kartı bilgisi gerektirmeyen ve öğrencilere süresiz ücretsiz katman sunan Render platformu seçilmiştir.
- **Konteyner Mimarisi (Containerization):** Ödevin “sanal makinelerin veya konteynerlerin oluşturulması” maddesine uygun olarak; sanal makine (VM) yöntemi yerine, modern DevOps süreçlerinde standartlaşan **Docker** tabanlı dağıtım modeli tercih edilmiştir.
- **Sürekli Dağıtım (CI/CD):** GitHub entegrasyonu sayesinde, kod deposuna yapılan her güncellemenin (push) otomatik olarak bulut ortamına yansması sağlanmıştır.

3. Uygulama Mimarisi Şeması

Uygulama, Docker konteyneri içerisinde izole bir şekilde çalışmaktadır. Veri tutarlılığı ve bulut maliyetlerini optimize etmek amacıyla uygulama katmanında **In-Memory Database** (Bellek içi Veritabanı) kullanılmıştır.

Mimari Akışı:

1. **İstemci:** Kullanıcı tarayıcıdan <https://projetakip-0ktd.onrender.com> adresine istek atar.
2. **Render Cloud:** Gelen isteği karşılar ve Load Balancer üzerinden ilgili servise yönlendirir.
3. **Docker Container:** Uygulama, Linux tabanlı bir konteyner içinde çalışır (ASP.NET Core Runtime).
4. **Veri Katmanı:** Uygulama verileri, çalışma zamanında bellek (RAM) üzerinde tutulur.



4. Uygulamanın Bulut Platformuna Taşınması

Uygulamanın yerel ortamdan buluta taşınması süreci aşağıdaki adımlarla gerçekleştirilmiştir:

Adım 1: Projenin Buluta Hazırlanması

Yerel ortamda SQL Server kullanan proje, bulut ortamında ekstra Veritabanı sunucu maliyeti oluşturmaması ve hızlı dağıtım yapılabilmesi için **In-Memory Database** yapısına geçilmiştir. **Program.cs** dosyasında gerekli bağımlılık enjeksiyonları yapılmıştır.

Adım 2: Dockerize İşlemi (Konteyner Oluşturma)

Uygulamanın her ortamda çalışabilmesi için kök dizine bir **Dockerfile** eklenmiştir. Bu dosya, uygulamanın nasıl derleneceğini (build) ve çalıştırılacağını (run) tarif eder.

```

Krane04 Update Dockerfile
5409168 · 2 hours ago · History

Code Blame 22 lines (15 loc) · 593 Bytes

1 FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
2 WORKDIR /src
3
4 COPY ["Erkan_aktunc_web/Erkan_aktunc_web.csproj", "Erkan_aktunc_web/"]
5
6 RUN dotnet restore "Erkan_aktunc_web/Erkan_aktunc_web.csproj"
7
8 COPY . .
9
10 WORKDIR "/src/Erkan_aktunc_web"
11
12 RUN dotnet build "Erkan_aktunc_web.csproj" -c Release -o /app/build
13
14 FROM build AS publish
15 RUN dotnet publish "Erkan_aktunc_web.csproj" -c Release -o /app/publish
16
17 FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
18 WORKDIR /app
19 COPY --from=publish /app/publish .
20 ENV ASPNETCORE_URLS=http://*:8080
21 EXPOSE 8080
22 ENTRYPOINT ["dotnet", "Erkan_aktunc_web.dll"]

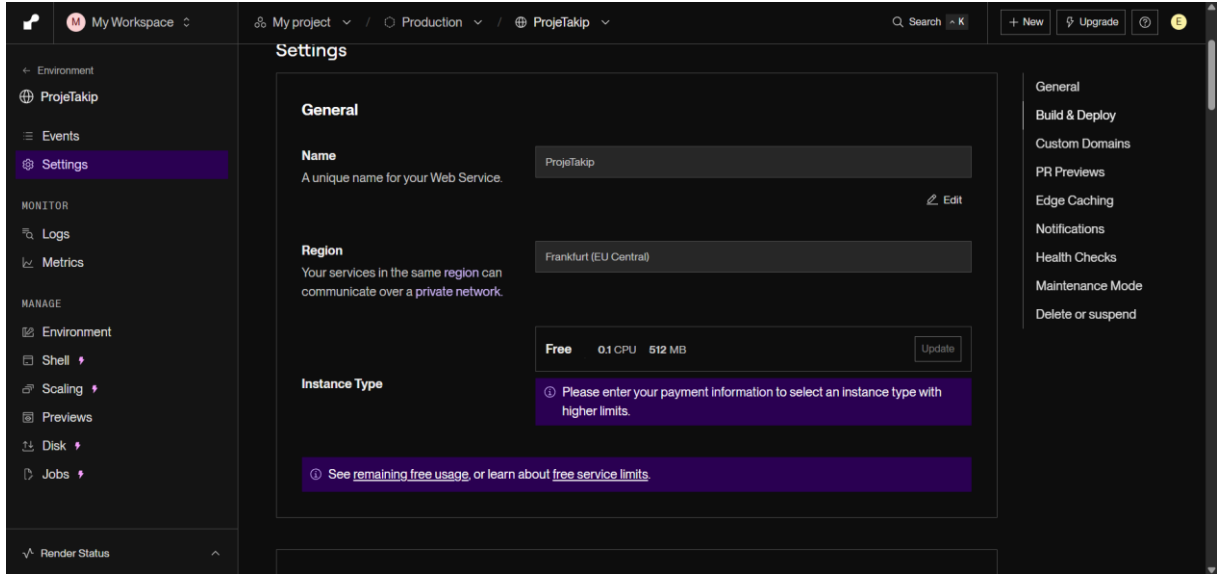
```

Adım 3: Bulut Servisinin Oluşturulması

Render.com üzerinde yeni bir “Web Service” oluşturulması ve GitHub deposu ile bağlantı sağlanmıştır.

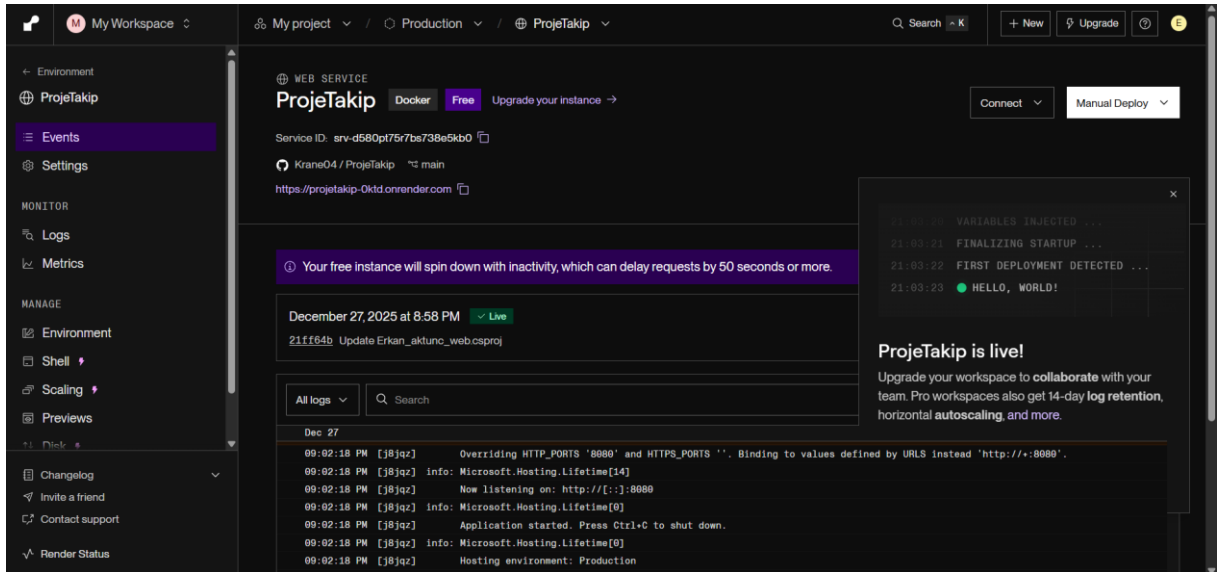
- **Repo:** [Krane04 / ProjeTakip](#)
- **Runtime:** Docker
- **Bölge:** Frankfurt (EU Central)

- **Plan: Free**



Adım 4: Dağıtım ve Çalıştırma

Yapılandırma tamamlandıktan sonra Render otomatik olarak Docker imajını derlemiş ve uygulamayı 8080 portu üzerinden yayına almıştır.



5. Karşılaşılan Zorluklar ve Çözümler

Projenin dağıtım sürecinde karşılaşılan teknik aksaklıklar ve uygulanan çözümler şunlardır:

1. **Dosya Yolu Hatası (File Not Found):** Dockerfile oluşturulurken proje dosyasının (**.csproj**) ana dizinde olduğu varsayılmıştır. Ancak Dosyanın bir alt klasörde olması nedeniyle “Build Failed” hatası alınmıştır.

- **Çözüm:** Dockerfile içerisindeki COPY ve RUN komutlarına klasör yolu eklenerek yol düzeltilmiştir.
2. **Framework Sürüm Uyuşmazlığı:** Docker imajı .NET 8.0 tabanlı iken, projenin hedef sürümünün yanlışlıkla .NET 10.0 olarak ayarlandığı tespit edilmiştir.
- **Çözüm:** .csproj dosyasındaki hedef sürüm `<TargetFramework>net8.0</TargetFramework>` olarak güncellenmiştir.
3. **Eksik Bağımlılık Hatası:** Kod tarafında `UseInMemoryDatabase` metodu kullanılmasına rağmen, ilgili kütüphanenin projede yüklü olmadığı görülmüştür.
- **Çözüm:** `Micorosoft.EntityFrameworkCore.InMemory` paketi projeye dahil edilerek sorun giderilmiştir.

6. Otomasyon Kodları (Dockerfile)

Uygulamanın otomasyonu ve konteynerizasyonu için kullanılan kod yapısı aşağıdadır:

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
```

```
WORKDIR /src
```

```
COPY ["Erkan_aktunc_web/Erkan_aktunc_web.csproj", "Erkan_aktunc_web/"]
```

```
RUN dotnet restore "Erkan_aktunc_web/Erkan_aktunc_web.csproj"
```

```
COPY . .
```

```
WORKDIR "/src/Erkan_aktunc_web"
```

```
RUN dotnet build "Erkan_aktunc_web.csproj" -c Release -o /app/build
```

```
FROM build AS publish
```

```
RUN dotnet publish "Erkan_aktunc_web.csproj" -c Release -o /app/publish
```

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
```

```
WORKDIR /app
```

COPY --from=publish /app/publish .

ENV ASPNETCORE_URLS=http://+:8080

EXPOSE 8080

ENTRYPOINT ["dotnet", "Erkan_aktunc_web.dll"]

7. Sonuç

Bu ödevin kapsamında, modern bir web uygulaması başarıyla konteynerize edilmiş ve bulut ortamında çalışır hale getirilmiştir. Bulut platformunun sunduğu CI/CD yetenekleri sayesinde, kod değişiklikleri canlı ortama aktarılması süreci otomatize edilmiştir. Uygulama şu anda <https://projetakip-0ktd.onrender.com> adresinde aktif olarak çalışmaktadır.

Linkler

Youtube Link: <https://youtu.be/vWJBFcSfNIc>

GitHub Link: <https://github.com/Krane04/ProjeTakip>